

Ex. 2.5 (20 pts) — Pseudo-code for fork, exec, waitpid, exit

Following the hint, each algorithm below is the *kernel-side implementation* of the call: the code the operating system runs when a user process traps into it. All four operate on the process table, whose entries are process control blocks (PCBs); `CURRENT` denotes the PCB of the calling process, and a negative return value is the usual `errno` convention.

Algorithm 1 `fork()` — create a copy of the calling process

```
1: procedure FORK()
2:   if no free process-table slot or no free pid then
3:     return -EAGAIN
4:   end if
5:    $c \leftarrow \text{ALLOCATEPCB}()$ ;  $c.\text{pid} \leftarrow \text{ALLOCATEPID}()$ 
6:   copy from CURRENT into  $c$ : uid, gid, cwd, root, umask,
     signal dispositions, resource limits, priority
7:    $c.\text{parent} \leftarrow \text{CURRENT}$ ; append  $c$  to CURRENT.children
8:    $c.\text{addrspace} \leftarrow \text{COPYONWRITESHARE}(\text{CURRENT}.\text{addrspace})$ 
9:   mark every writable page read-only in both processes ▷ copy on first write
10:  for all descriptors  $fd$  in CURRENT.fdtable do
11:     $c.\text{fdtable}[fd] \leftarrow \text{CURRENT.fdtable}[fd]$ 
12:    increment the reference count of that open-file entry
13:  end for
14:   $c.\text{cputime} \leftarrow 0$ ;  $c.\text{pending signals} \leftarrow \emptyset$ 
15:   $c.\text{savedregs} \leftarrow \text{CURRENT.savedregs}$  ▷ child resumes at the instruction after the trap
16:   $c.\text{savedregs.retval} \leftarrow 0$  ▷ what the child will see fork return
17:   $c.\text{state} \leftarrow \text{READY}$ ; ENQUEUE(ready queue, c)
18:  return  $c.\text{pid}$  ▷ what the parent will see fork return
19: end procedure
```

The two return-value assignments — 0 into the child’s saved register, the child’s pid into the parent’s — are the whole “`fork` returns twice” phenomenon: one call site, two saved register sets, two different values in the return register.

Algorithm 2 `exec(path, argv, envp)` — replace the caller’s program image

```
1: procedure EXEC(path, argv, envp)
2:   inode ← NAMEI(path); if inode = NULL then return -ENOENT
3:   if inode is not executable by CURRENT.uid then return -EACCES
4:   hdr ← READHEADER(inode)
5:   loader ← LOADERFOR(hdr.magic)           ▷ magic number selects a.out / ELF / script
6:   if loader = NULL then return -ENOEXEC
7:   kargv, kenvp ← COPYINTOKERNEL(argv, envp)           ▷ they live in the old address space
   — point of no return: every error above is still reportable to the caller —
8:   DESTROYADDRESSSPACE(CURRENT.addrspace)
9:   CURRENT.addrspace ← loader.MAPIMAGE(inode)           ▷ text r-x, data rw, bss zeroed
10:  create a fresh heap and user stack
11:  for all signals s caught by a user handler do
12:    disposition of s ← DEFAULT           ▷ the handler’s code is gone
13:  end for
   ignored dispositions and the blocked signal mask are left untouched
14:  close every descriptor marked close-on-exec
15:  if inode is set-uid then CURRENT.euid ← inode.owner
16:  sp ← PUSHONTONEWSTACK(kargv, kenvp)
17:  CURRENT.savedregs ← { pc : hdr.entry, sp : sp, others : 0 }
18:  return to user mode           ▷ on success there is no caller left to return to
19: end procedure
```

Algorithm 3 `waitpid(pid, statusp, options)` — collect a terminated child

```
1: procedure WAITPID(pid, statusp, options)
2:   loop
3:     found ← FALSE
4:     for all c ∈ CURRENT.children matching pid do           ▷ pid = -1 matches any child
5:       found ← TRUE
6:       if c.state = ZOMBIE then
7:         if statusp ≠ NULL then COPYOUT(statusp, c.exitstatus)
8:         CURRENT.childcputime += c.cputime + c.childcputime
9:         remove c from CURRENT.children
10:        p ← c.pid; FREEPCB(c); FREEPID(p)           ▷ the reaping step
11:        return p
12:      end if
13:    end for
14:    if found = FALSE then return -ECHILD
15:    if options contains WNOHANG then return 0
16:    if a signal is pending for CURRENT then return -EINTR
17:    SLEEP(CURRENT, its wait channel)           ▷ a child’s exit wakes us; then rescan
18:  end loop
19: end procedure
```

Algorithm 4 `exit(status)` — terminate the calling process

```
1: procedure EXIT(status)
2:   for all open descriptors fd of CURRENT do
3:     CLOSE(fd)                                ▷ drops the reference count; the last close releases the file
4:   end for
5:   RELEASEADDRESSSPACE(CURRENT.addrspace)
6:   release cwd, root, timers and file locks
7:   for all c ∈ CURRENT.children do
8:     c.parent ← init; move c to init.children
9:     if c.state = ZOMBIE then SIGNAL(init, SIGCHLD); WAKEUP(init)    ▷ so init reaps it
10:  end for
11:  CURRENT.exitstatus ← status; record CPU time and accounting in the PCB
12:  CURRENT.state ← ZOMBIE                                ▷ PCB and pid stay allocated until reaped
13:  SIGNAL(CURRENT.parent, SIGCHLD); WAKEUP(CURRENT.parent)
14:  SCHEDULE()                                             ▷ picks another process; exit never returns
15: end procedure
```

How the four calls are inter-related

They are not four independent services but two pairs that bracket the life of a process.

`fork` is the only way a Unix system makes a new process, and `exec` is the only way a process changes the program it runs. Neither alone can start a new program: `fork` produces a second copy of the caller, `exec` replaces the caller's own image. Running a new program is therefore necessarily both of them, `fork` then `exec` in the child. The two algorithms are built to meet: `fork` gives the child a duplicate descriptor table and the parent's credentials, and `exec` keeps exactly that table, less the close-on-exec entries, while discarding everything else.

The split is what makes the arrangement useful. Between `fork` returning 0 and `exec` overwriting the image there is a window in which the child is still running the parent's code and can rearrange precisely the state that `exec` preserves. That window is where a shell implements `ls > out`: the child closes descriptor 1, reopens it on the file, then execs `ls`, which need know nothing about redirection.

`exit` and `waitpid` are the matching pair at the other end, and they are a pair for one reason: the exit status has to outlive the process that produced it. `exit` can free the address space and the descriptors, but it cannot free the PCB, because nobody has read the status out of it yet; so it stores the status there, marks the process ZOMBIE and signals the parent. `waitpid` is the other half of that transaction: it finds the zombie child, copies the status out, charges its CPU time to the parent, and only then releases the PCB and the pid. Until then the pid stays reserved, so "wait for process *p*" stays unambiguous.

A shell's main loop is the whole mechanism in one cycle: `fork`; the child `execs` the command; the parent blocks in `waitpid`; the child `exits` and becomes a zombie; the parent's `waitpid` returns with the status, frees the PCB, and prints the next prompt.

Ex. 2.6 (5 pts) — Zombie processes

A zombie is a process that has already run `exit()` but has not yet been reaped. Its address space, open files and kernel stack are gone; all that is left is its process-table entry, holding the exit status, the pid and the CPU time it used.

The kernel keeps that entry because the status belongs to the parent, and the child cannot report it once it no longer exists. The pid must stay allocated for the same reason: reusing it while the parent can still name it in `waitpid` would let the parent wait on an unrelated process.

The cost is one process-table slot and one pid — no memory, no CPU time, no descriptors. A zombie also cannot be killed, since it is already dead: `SIGKILL` is discarded, as no execution context is left to receive it. In `ps` it appears in state Z, with its command shown as `<defunct>`.

It disappears when the parent calls `wait` or `waitpid`, or when the parent itself exits and `init` inherits and reaps it. A long-lived parent that forks children and never waits leaks table entries until `fork` fails with `EAGAIN`, which is why such a parent should handle or explicitly ignore `SIGCHLD`.

An orphan is the complement: a child still *alive* when its parent dies. `init` adopts it and reaps it when it eventually exits.